

Robust Manifold Segmentation via Entropic Wasserstein Propagation

Roe Ben-Shlomo
roeebs@mail.tau.ac.il

January 22, 2026

Abstract

This project explores the application of optimal transport and heat diffusion for robust mesh segmentation. We implement a Wasserstein propagation algorithm based on Solomon et al.’s [2] heat method. We demonstrate that while naive Euclidean segmentation can segment an easy & symmetric human mesh, it fails on meshes that are more complex topologically, while our geodesic-based approach maintains consistency. We further analyze the impact of entropic regularization (γ) on convergence stability and propose a spectral-based future research direction to automate initialization.

1 Theoretical Background

This small project is based on the paper ”Convolutional Wasserstein Distances: Efficient Optimal Transportation on Geometric Domains” by Solomon et al. (2015). The paper addresses a fundamental computational bottleneck in geometry processing: computing Optimal Transport distances on curved domains, which traditionally requires solving a linear program that’s $O(N^3)$.

Like the paper, we’ll use M as our compact, Riemannian manifold which represents a surface (or image plane in the paper). We use $d : M \times M \rightarrow \mathbb{R}_+$ to denote the geodesic distance function, so $d(x, y)$ is the shortest distance from x to y along M . We’ll also use $\text{Prob}(M)$ to indicate the space of probability measures on M .

1.1 The Optimal Transport Problem

The 2-Wasserstein distance provides a meaningful metric for comparing two probability distributions, μ and ν , defined on our manifold M . Unlike point-wise metrics like L_2 or KL-Divergence, Wasserstein distance accounts for the underlying geometry of the domain. It measures the minimum ”work” required to transport mass from μ to ν :

$$\mathcal{W}_2^2(\mu, \nu) = \inf_{\pi \in \Pi(\mu, \nu)} \iint_{M \times M} d(x, y)^2 d\pi(x, y) \quad (1)$$

Where $\Pi(\mu, \nu)$ is the set of joint distributions (couplings) whose marginals are μ and ν . Solving this directly involves a large scale Linear Programming problem, which scales cubically, it computationally intractable for detailed 3D meshes where we have thousands, if not tens of thousands of vertices.

1.2 Entropic Regularization

To overcome this computational obstacle, the paper leverages entropic regularization. By adding an entropy penalty $H(\pi)$ to the transport cost, the problem becomes strictly convex:

$$\mathcal{W}_{2,\gamma}^2(\mu, \nu) = \inf_{\pi \in \Pi} \left[\iint_{M \times M} d(x, y)^2 \pi(x, y) dx dy - \gamma H(\pi) \right] \quad (2)$$

The parameter γ controls the trade-off between precision and diffusion. As $\gamma \rightarrow 0$ the solution approaches the true Wasserstein distance, and as $\gamma \rightarrow \infty$ the transport plan becomes infinitely diffuse yet easier to be calculated. This regularization allows the optimal plan to be found using the Sinkhorn Algorithm, which is an iterative method involving simple matrix-vector multiplications, and is detailed in the original paper.

1.3 The Convolutional Insight

The central contribution of Solomon et al. is overcoming the quadratic complexity of computing the transport plan. In the standard regularized formulation, the distance kernel $\mathcal{K}_\gamma(x, y) = e^{-d(x, y)^2/\gamma}$ requires computing all-pairs of the geodesic distances, which is $O(N^2 \log N)$ or worse. The authors utilize Varadhan’s formula which links the geodesic distance to the heat kernel $\mathcal{H}_t$:

$$d(x, y)^2 = \lim_{t \rightarrow 0} [-2t \ln \mathcal{H}_t(x, y)] \quad (3)$$

By setting the diffusion time to $t = \gamma/2$ the distance kernel can be approximated by the heat kernel:

$$\mathcal{K}_\gamma(x, y) \approx \mathcal{H}_{\gamma/2}(x, y) \quad (4)$$

This substitution is significant: instead of computing explicit distances, we can perform ”convolution”

by solving the heat diffusion equation. This reduces the problem to sparse linear algebra, which scales linearly $O(N)$ with the mesh size.

1.4 The Heat Method Pipeline

To implement the heat kernel operator H_t on a discrete triangular mesh, we utilize the cotangent Laplacian $L \in \mathbb{R}^{n \times n}$ and the lumped area matrix D_a (a diagonal matrix where D_{ii} is the area associated with vertex i).

The convolution of a signal v against the heat kernel, denoted in the paper as $\mathbf{w} = H_t(\mathbf{a} \otimes \mathbf{v})$, corresponds to solving the implicit Euler step for the heat equation:

$$(D_a + tL)\mathbf{w} = \mathbf{a} \otimes \mathbf{v} \quad (5)$$

Where $\otimes$ denotes element-wise multiplication. Since $D_a + tL$ is sparse and positive-definite, it can be pre-factored with Cholesky decomposition, making repeat applications extremely fast.

1.5 Application to Segmentation

We model mesh segmentation as a label propagation problem via Optimal Transport. We define a set of sparse anchor vertices, each assigned a class label. We treat these anchors as the source distribution μ (Dirac deltas at seed locations) and the entire mesh geometry as the target distribution ν (a uniform distribution over the manifold).

By setting ν to be uniform, we effectively ask the solver to transport the mass from the sparse seeds to "fill" the entire surface of the mesh. We solve for the optimal transport plan $\pi \in \mathbb{R}^{n \times k}$ using the Sinkhorn Algorithm (Algorithm 1 in the paper). The entry π_{ik} represents the amount of mass vertex i receives from class k . We obtain the final segmentation by assigning each vertex to the class that contributes the most mass to it: $\text{Label}(i) = \arg \max_k \pi_{ik}$.

2 Implementation & Results

We implemented the Convolutional Wasserstein framework in Python using `scipy.sparse` for the linear solvers and `PyVista` for visualization. The algorithm was tested on the human poses from the SCAPE dataset.

2.1 Experiments Setup

We initialized the solver with 7 seed points (anchors) placed on key body parts: head, hands, feet and torso. The source distribution μ was initialized as a sum of Diracs at these seed locations. The target ν was set to a uniform distribution over the manifold, which in our discrete setting is implemented as an area-weighted vector to account for varying mesh element sizes. We utilized the Heat method with



Figure 1: The Convolutional Wasserstein Segmentation pipeline. Given a small set of anchor points (white spheres) representing class labels, our algorithm propagates these labels across the manifold using entropic optimal transport, resulting in a smooth, topology-aware segmentation.

the cotangent Laplacian to approximate the transport kernel, enabling efficient computation even on this dense mesh.

2.2 Experiment #1: Topological Consistency

We compared our Geodesic (Wasserstein) approach against a naive Euclidean approach (Nearest Neighbors in 3D space). To quantify the performance, we defined a "Geodesic Violation Score" metric:

$$V(v) = \max(0, d(v, s_{\text{assigned}}) - d(v, s_{\text{closest}})) \quad (6)$$

This metric quantifies the error: how far a vertex is from its assigned seed compared to the true geometrically closest seed. Ideally, this score should be zero (a perfect Voronoi partition on the manifold).

As seen in Figure 2, the naive Euclidean approach fails at the interface where the hand is close to the knee and when the arm is close to the hip. Because the Euclidean distance is short, it incorrectly labels the knee as a hand (yellow). This results in a high violation score (Max 1.43). In contrast, our Wasserstein approach diffuses the probability along the surface (up the arm, across the shoulder, down the torso), correctly identifying the distinct topology. The violation score remains negligible (Max 0.13).

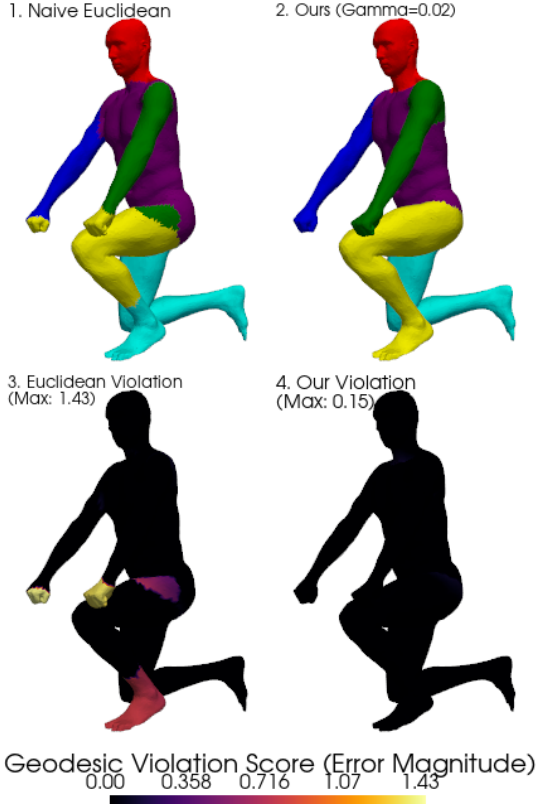


Figure 2: Topological consistency. Top: Euclidean segmentation (left) violates topology at the hand/knee, while our method (right) is consistent. Bottom: Geodesic Violation scores.

2.3 Experiment #2: Stability vs. Regularization

We analyzed the impact of the entropic regularization parameter γ . The results in Figure 3 demonstrate the theoretical trade-off inherent in Entropic Optimal Transport. We observe 3 distinct behavioral regimes:

- **Numerical instability** ($\gamma = 1e - 06$): as $\gamma \rightarrow 0$, the problem approaches the unregularized Wasserstein distance. However, the heat kernel values approach zero, leading to numerical underflow and division by zero within the Sinkhorn iterations.
- **Working range** ($\gamma \in [0.002, 0.1]$): In this sweet spot, the segmentation is stable. Lower values here yield sharper boundaries, while higher values increase diffusion, providing a soft segmentation confidence map.
- **Over-Regularization** ($\gamma = 10.0$): When γ is too large, the entropy term dominates the transport cost. The mass spreads excessively, making the segmentation worse. The average confidence drops to 0.18, indicating a less-informative transport plan.

3 Future Research

A major limitation of our current implementation is its reliance on manual initialization. We propose automating this via Spectral geometry, utilizing the eigenfunctions of the Laplace-Beltrami operator that is already pre-factored in our pipeline.

Proposal: Spectral Seeding We hypothesize that the semantic centers required for segmentation coincide with the extrema of the low-frequency eigenfunctions of L . We propose 2 alternatives to detect them:

Alternative A: Spectral Clustering Initialization Since we already compute the Cholesky factorization of $D_a + tL$, extracting eigenvectors via an iterative solver is efficient. *Algorithm:* (1) Compute the first k eigenpairs (λ_i, ϕ_i) of L , (2) Embed vertices into $\mathbb{R}^k$ via spectral coordinates, (3) Perform KMeans clustering in this embedding (4) Map centroids back to the mesh as seeds.

Alternative B: Deep Spectral Seeding (DiffusionNet) This approach replaces unsupervised clustering with a supervised learning model, inspired by the architecture of DiffusionNet [1]. Since our current implementation already computes $\mathcal{H}_t$, we can easily generate the Heat Kernel Signatures for every vertex. Unlike raw coordinates, HKS features are invariant to isometric deformations. A DiffusionNet can take these features to learn as a semantic prior. *Algorithm:* (1) Compute HKS features (2) Train a vertex classifier to minimize CrossEntropy loss against GT seed maps (3) select local maxima of the predicted probability field.

3.1 Research Plan & Methodology

To conduct this future research, we define the following methodology: **(1) Data Preparation:** We will utilize the SCAPE dataset to generate synthetic training pairs (Mesh, SeedMap) for the supervised model, placing Gaussians at the ground-truth segment centers as a sort of label smoothing. **Implementation:** We will use `scipy.sparse` to extract the bottom k eigenvectors of our Laplacian matrix L and to compute the HKS features. We will integrate these with a PyTorch implementation of DiffusionNet. **Validation & Stability Analysis:** We will define a "Seed Stability" metric by applying random isometric deformations to the mesh and measure the geodesic deviation of the automatically selected seeds across different poses. Our hypothesis is that the

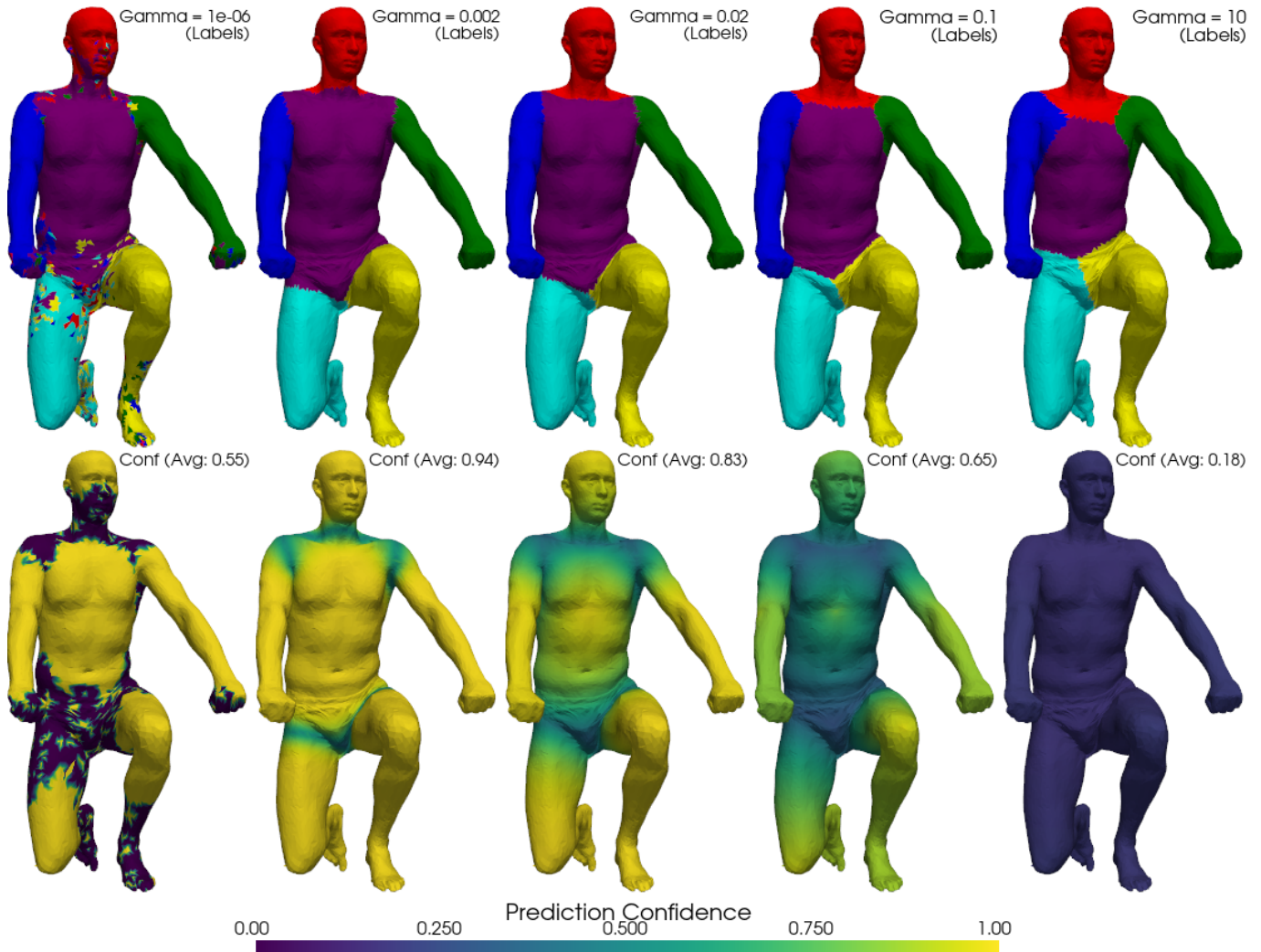


Figure 3: Sensitivity to Entropic Regularization (γ). Top: hard segmentation labels. Bottom: prediction confidence ($\max_k \pi_{ik}$). Low γ yields sharp boundaries as long as we’re numerical stable, high γ leads to excessive diffusion, demonstrating the theory of entropic Optimal Transport.

spectral seeds will move coherently with the shape deformation, where Euclidean heuristics would fail.

4 Conclusion

We successfully implemented the Convolutional Wasserstein Distance framework and applied it to the problem of 3D mesh segmentation. We demonstrated that by approximating the geodesic distance kernel via heat diffusion, we can solve the Optimal Transport problem efficiently on curved domains. Our results confirm that this method is topologically robust where Euclidean methods fail, and that the regularization parameter γ serves as a tunable “temperature” controlling the trade-off between geometric precision and algorithmic stability.

5 AI Usage

I used AI for several things: project ideas brainstorming and feasibility, PyVista plotting issues like rain-

bows and shared colorbars, debugging issues, the usage of the dijkstra algorithm, rephrasing my document to shorten from 5 to 4 pages, and latex issues.

References

- [1] N. Sharp, S. Attai, K. Crane, and M. Ovsjanikov. Diffusionnet: Discretization agnostic learning on surfaces, 2022.
- [2] J. M. Solomon, F. de Goes, G. Peyré, M. Curi, A. Butscher, A. T. Nguyen, T. Du, and L. J. Guibas. Convolutional wasserstein distances. *ACM Transactions on Graphics (TOG)*, 34:1–11, 2015.